

Global Grid

Objectives :

- 2D map of Robots env
- C-space problem
- RASTERIZATION

2D MAP :



CONTINUOUS SPACE

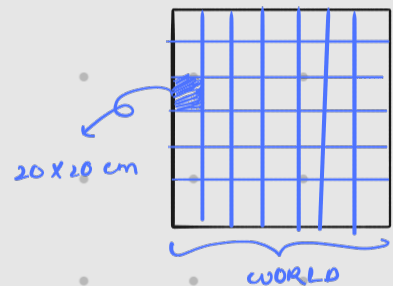
CONTINUOUS SPACE



DISCRETIZED SPACE → divide the WORLD into squares

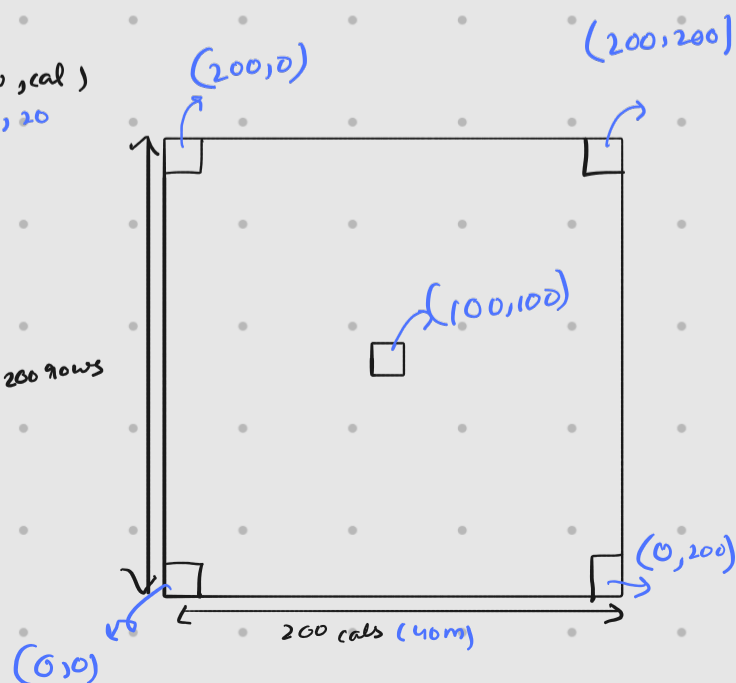
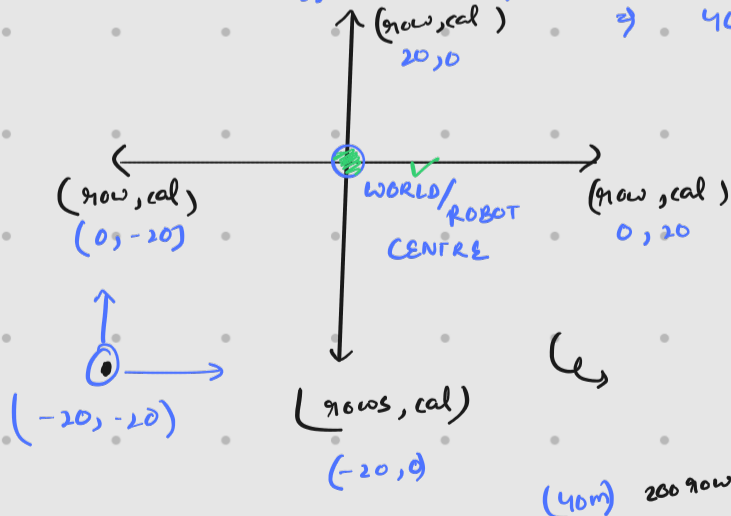
eg: resolution = 0.2

∴ Every square → 20cm x 20cm



grid size = 200

∴ Total map size → 200 x 0.2
→ 40m



$$\# \text{ row} = (y - y_{\text{origin}}) / \text{resolution}$$

$$\# \text{ col} = (x - x_{\text{cal}}) / \text{resolution}$$

y : robot: $\begin{pmatrix} y & x \\ \text{row, col} \\ 1.7\text{m} & 3.4\text{m} \end{pmatrix}$

$$\begin{aligned} \rightarrow \text{row} &= (y - y_{\text{origin}}) / \text{resolution} \\ &= (1.7 - (-20)) / 0.2 \\ &= 108.5 \rightarrow \text{int}(108.5) = 108 \end{aligned}$$

$$\begin{aligned} \rightarrow \text{col} &= (x - x_{\text{origin}}) / \text{resolution} \\ &= (3.4 - (-20)) / 0.2 \\ &= 117 \rightarrow \text{int}(117) = 117 \end{aligned}$$

Robot cell = (108, 117)

BACK TO WORK from GRID^a

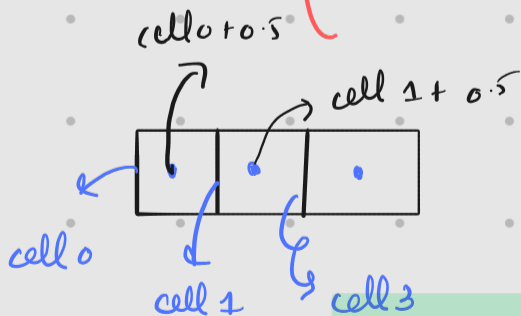
$\rightarrow$ Robot cell = (108, 117)

$$\rightarrow \text{row} = (y - y_{\text{origin}}) / \text{resolution}$$

$$\begin{aligned} & \left\{ \begin{aligned} & (\text{row} \times \text{resolution}) + y_{\text{origin}} = y \\ & \rightarrow (108 \times 0.2) + (-20) = y \\ & \rightarrow 21.6 - 20 = \boxed{y = 1.6} \end{aligned} \right\} \end{aligned}$$

BUG

$\downarrow$
use calculate from middle of cell



$\downarrow$ corrected approach

$$\begin{aligned} \rightarrow y &= \text{origin}_y + (\text{row} + 0.5) \times \text{resolution} \\ &\rightarrow y = -20 + (108 + 0.5) \times 0.2 \\ &= -20 + 21.7 \\ &= 1.7 \end{aligned}$$

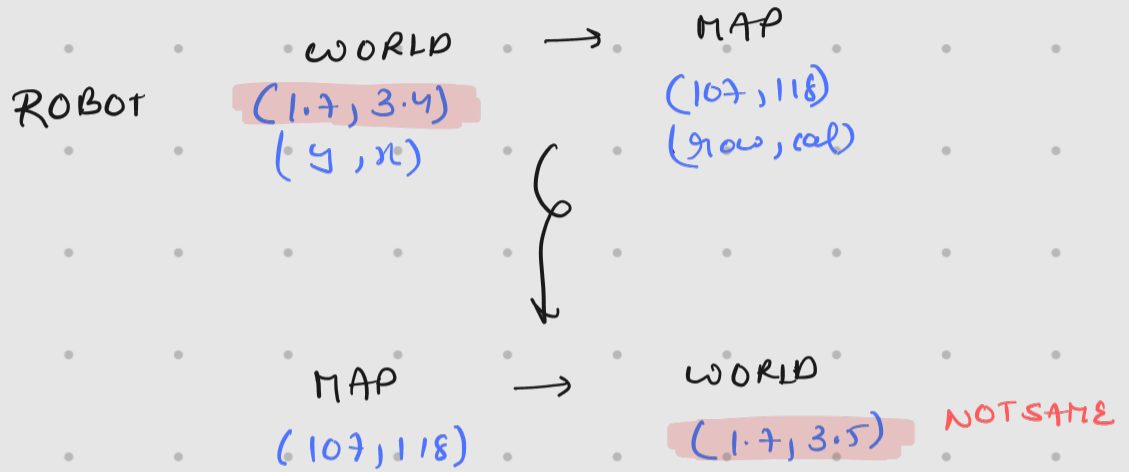
✓
CORRECTED APPROACH

$x = x_{axis} + (cal + 0.5) \times resolution$

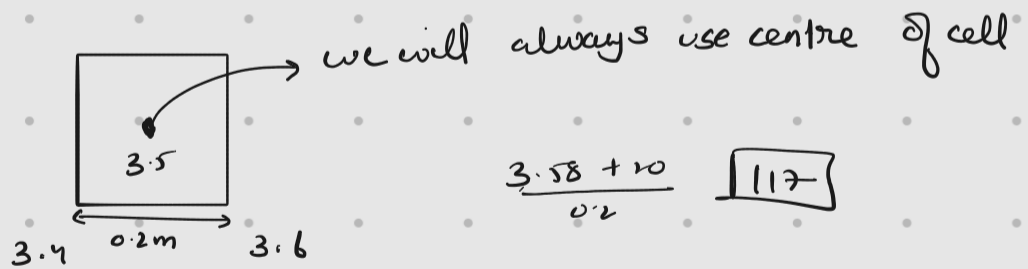
$x = -20 + (117 + 0.5) \times 0.2$
 $= -20 + 23.5$
 $= 3.5$

$$\begin{array}{r} 117.5 \\ \times 2 \\ \hline 2350 \end{array}$$

NOTICE



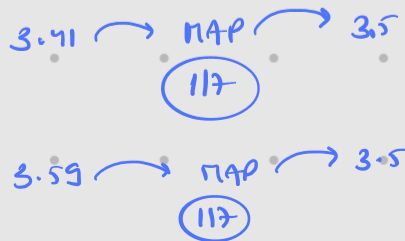
ROLE of QUANTIZATION ERROR.



$$\frac{3.58 + 10}{0.2} = 117$$

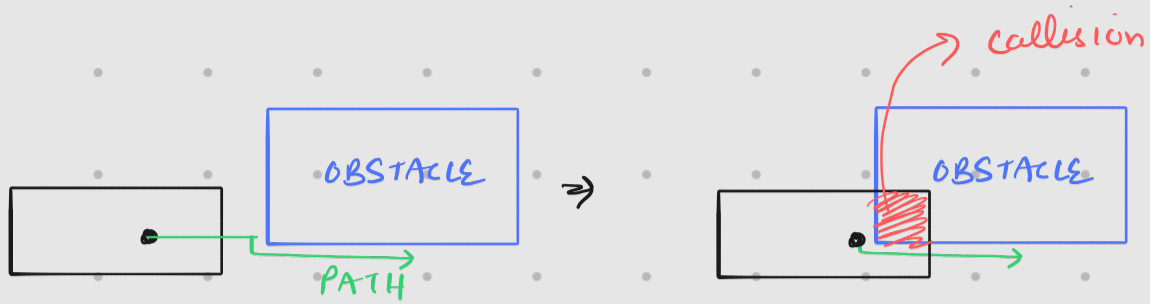
Quantization error = $\frac{resolution}{2} = \frac{0.2}{2} = 0.1m = 10cm$

eg



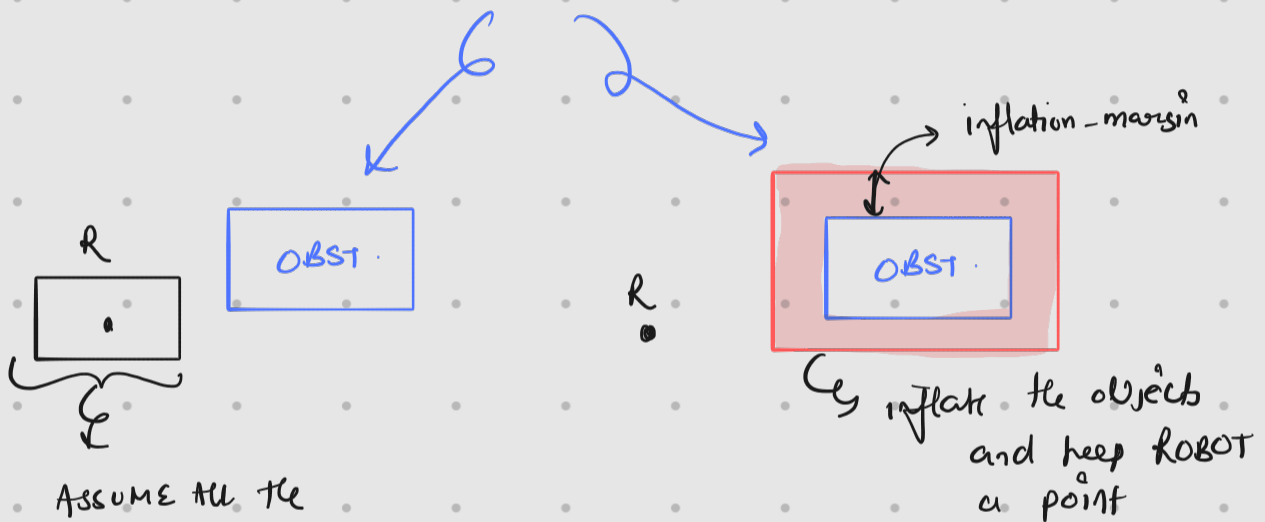
centre $\therefore$ max of $10cm$ of error

C-Space $\rightarrow$ Configuration Space



$\therefore$ checking only the centre can cause collisions
 $\therefore$ WRONG

SOLUTIONS



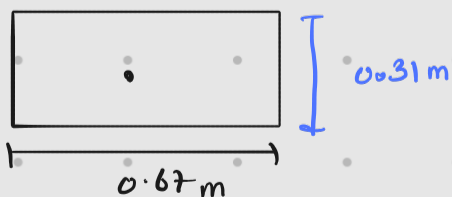
Assume all the cells for A*

COMPUTATIONALLY EXPENSIVE

SAME A* simple

$$\# \text{ Inflation Margin} = \text{Robot half width} + \text{Safety Margin}$$

OUR CASE $\rightarrow$ UNITREE GO 2



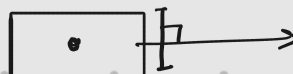
$$\therefore \text{Robot half width} = 0.155$$

$$\text{safety margin} = 0.145$$

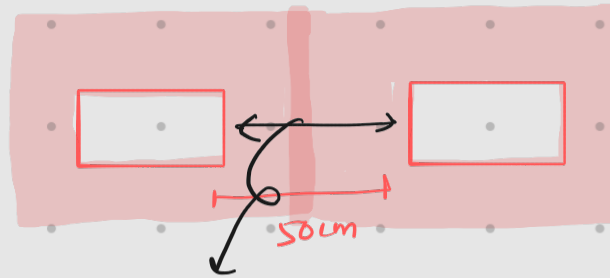
$$\therefore \text{INFLATION MARGIN} = 0.30$$

why not diagonal / length in play.

1) Robot width is perpendicular to movement



2) if we introduce diagonal $\rightarrow$

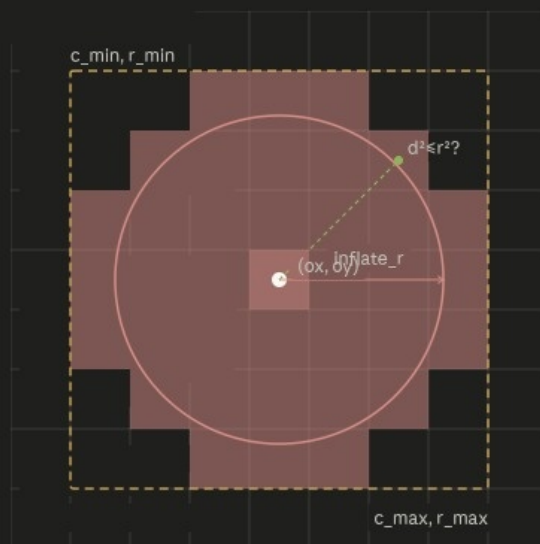


despite of free space robot to walk
 $\rightarrow$ inflation blocks the space.

$\therefore$ we should not be much conservative.

##

Finally, the **rasterization circle check** — how each inflated obstacle gets stamped onto the grid:



the two-step trick

① bounding box scan
compute c_min/max ,
 r_min/max from circle
extents. Avoids iterating
the whole 200×200 grid.
 $O(r^2)$ not $O(N^2)$.

② circle membership test
For each cell in box,
check cell center (cx, cy)
against the circle. No
sqrt needed — compare
 d^2 vs r^2 directly.

$(cx - ox)^2 + (cy - oy)^2$
 $\leq inflate_r^2$
 $\rightarrow$ mark cell as occupied

Intuition behind the two-step rasterization:

The naive approach — check every cell in the 200×200 grid — wastes time on cells far from the obstacle. Instead, the code first computes a tight **bounding box** (the orange dashed rectangle) around the inflated circle using simple min/max arithmetic. Only cells within this box are tested. That reduces work from 40,000 cells to roughly $\pi \times r^2$ cells per obstacle.

Then for each candidate cell, it computes the **squared distance** from the cell's center (cx, cy) to the obstacle center (ox, oy) and compares it to $inflate_r^2$. This avoids an expensive square root — you only need the square root if you want the actual distance, but for a $\leq$ comparison, both sides can stay squared.